UNIVERSIDADE DE FORTALEZA

Fundação Edson Queiroz

Curso de Engenharia da Computação

RELATÓRIO DE ATIVIDADE PRÁTICA: AV2

Uso do LCD Shield 16x2 com Teclado, Serial e Acelerômetro ADXL345 na Placa NUCLEO-STM32F446RE

Disciplina: Sistema de Tempo Real (STR)

Professor: Imbiriba

Aluno: Rafael Marins

Fortaleza, Novembro de 2026

1. Introdução

Interfaces homem-máquina simples são uma necessidade recorrente em sistemas embarcados: um operador precisa ler mensagens do sistema e fornecer comandos, muitas vezes sem recorrer a um computador. Um display de cristal líquido (LCD) alfanumérico de 16 colunas por 2 linhas, baseado no controlador HD44780, combinado a um teclado de poucas teclas, é uma das soluções mais difundidas para esse fim e está disponível na forma de um shield no padrão Arduino, compatível com os conectores da placa NUCLEO-STM32F446RE.

Este relatório documenta o desenvolvimento da atividade AV2: o acionamento do LCD 16x2 no modo de 4 bits, a leitura e a decodificação das teclas do shield por meio do conversor analógico-digital (ADC1), uma base de tempo gerada pelo temporizador TIM2 e usada em todos os atrasos, e a exibição, na segunda linha do display, de um caractere recebido pela interface serial (USART2). A programação foi feita em linguagem C com acesso direto aos registradores (sem uso da biblioteca HAL), seguindo a metodologia adotada nas Práticas 1 a 4 da disciplina.

Como parte adicional da atividade, o sistema recebe pela serial o comando ASCII "A" e responde com a leitura dos três eixos de um acelerômetro digital ADXL345, ligado à placa pelo barramento I^2C (I2C1). O acelerômetro mede a aceleração em cada eixo, em múltiplos da gravidade (g), e é um sensor de uso comum na detecção de inclinação, movimento e choque.

Além da implementação em si, o relatório registra duas dificuldades encontradas durante o desenvolvimento, a calibração das faixas do teclado a partir de medições reais e a espera por estouro do temporizador na rotina de atraso curto, e como cada uma foi resolvida.

2. Objetivos

Objetivo geral: desenvolver, em C e com acesso direto aos registradores, um sistema na placa NUCLEO-STM32F446RE que controla o LCD 16x2 do shield em modo de 4 bits, identifica a tecla pressionada por leitura analógica, mostra no display o nome da tecla e um caractere recebido pela interface serial e, ao receber o comando serial "A", lê os três eixos de um acelerômetro ADXL345 e os envia de volta pela serial.

Objetivos específicos:

- Configurar os pinos do LCD (E, RS, D4 a D7) como saídas e implementar o envio de nibbles, comandos e dados ao controlador HD44780, com a sequência de inicialização no modo de 4 bits;
- Implementar as funções de limpeza do display, de posicionamento do cursor por linha e coluna e de envio de strings;
- Implementar uma base de tempo de 1 µs com o TIM2, sem interrupção, e derivar dela os atrasos em microssegundos e em milissegundos;
- Ler o divisor resistivo do teclado pelo ADC1 e decodificar as teclas SELECT, LEFT, UP, DOWN e RIGHT a partir de faixas obtidas por medição;
- Receber um caractere pela USART2 sem bloquear o laço principal e exibi-lo na linha 2, coluna 7 do LCD;
- Configurar o I2C1 e implementar a leitura dos registradores do acelerômetro ADXL345, incluindo a identificação do sensor pelo registrador DEVID;

- Responder ao comando serial "A" com os valores dos três eixos, em unidades brutas e em mg;
- Validar o funcionamento do sistema na placa física, incluindo a calibração do teclado.

3. Revisão Bibliográfica

O HD44780 é o controlador de LCDs de caracteres mais comum no mercado. Ele possui memória interna para os caracteres exibidos (DDRAM), gerador de caracteres e lógica de controle, e é acessado por um barramento paralelo com os sinais RS (seleção de registrador: 0 para comando, 1 para dado), RW (leitura ou escrita) e E (habilitação), além das linhas de dados D0 a D7 [2]. O controlador captura os dados na borda de descida do sinal E, cujo pulso deve ter largura mínima de 450 ns, conforme o enunciado da atividade [4]. No modo de 4 bits, apenas D4 a D7 são ligados e cada byte é enviado em dois nibbles, o mais significativo primeiro. Como o display pode estar em modo de 8 bits ao ser energizado, a inicialização por software envia o meio byte 0x3 três vezes, com esperas mínimas de 4,1 ms e 100 µs, antes do nibble 0x2, que o coloca em modo de 4 bits [2]. A maioria dos comandos leva cerca de 37 µs; Clear Display e Return Home levam até 1,52 ms [4].

O teclado do shield usa uma rede resistiva em que cada tecla seleciona um ponto diferente de um divisor de tensão ligado ao 5 V, de modo que a tensão no ponto A0 é função da tecla pressionada; isso permite identificar cinco teclas com um único canal do ADC [4]. O ADC do STM32F4 tem 12 bits e converte 0 a 3,3 V em valores de 0 a 4095; tensões acima da referência saturam em 4095. O tempo de amostragem de cada canal é configurável no registrador ADC_SMPR2, e fontes de alta impedância exigem tempos maiores [1].

O temporizador TIM2 é de uso geral e de 32 bits: seu contador (CNT) é incrementado a cada ciclo do clock dividido pelo prescaler (PSC) e, sem interrupção habilitada, pode ser lido diretamente por software, o que permite medir intervalos de tempo por diferença de leituras [1].

A USART2 está no barramento APB1. Na placa NUCLEO, seus pinos PA2 (TX) e PA3 (RX), na função alternativa AF7, são ligados ao ST-LINK, que os apresenta ao computador como uma porta serial virtual [3]. A recepção pode ser verificada por varredura (polling) da flag RXNE do registrador USART_SR, que é limpa pela leitura de USART_DR [1].

O ADXL345 é um acelerômetro digital de três eixos, de baixo consumo, com resolução de 13 bits e faixa selecionável de ±2 g a ±16 g. Com o bit FULL_RES ativado, a sensibilidade permanece em 3,9 mg por LSB em qualquer faixa. Ele é acessado por I²C ou SPI: com o pino CS em nível alto, opera em I²C, e o pino SDO escolhe o endereço de 7 bits (0x53 com SDO em nível baixo, 0x1D com SDO em nível alto). O registrador DEVID (0x00) sempre retorna 0xE5, o que permite confirmar a presença do sensor; o bit Measure do registrador POWER_CTL (0x2D) tira o dispositivo do modo de espera; e os dados dos três eixos ficam em seis registradores consecutivos a partir de DATAX0 (0x32), em complemento de dois, com o byte menos significativo primeiro, podendo ser lidos em sequência com auto-incremento do endereço [5]. Em repouso, o sensor mede a aceleração da gravidade, de modo que o módulo do vetor (X, Y, Z) deve resultar em cerca de 1 g, qualquer que seja a orientação.

O periférico I2C1 do STM32F4 está no barramento APB1; seus sinais SCL e SDA podem ser levados aos pinos PB8 e PB9 pela função alternativa AF4, configurados como open-drain com resistores de pull-up. No modo padrão (100 kHz), o registrador I2C_CCR é calculado a partir do clock do APB1 e o I2C_TRISE

define o tempo máximo de subida dos sinais. A leitura de mais de dois bytes em modo mestre-receptor segue uma sequência específica de desligamento do ACK, STOP e leitura do registrador de dados, descrita no manual de referência [1].

4. Metodologia

4.1 Ambiente de desenvolvimento

O desenvolvimento foi realizado no STM32CubeIDE 2.2.0, no projeto av2_shield_lcd_serial, configurado para o microcontrolador STM32F446RETx (placa NUCLEO-F446RE), com a mesma abordagem bare-metal (registradores, sem HAL) empregada nas práticas anteriores. O código foi organizado em módulos: lcd.c/lcd.h (driver do display), tim.c/tim.h (base de tempo e atrasos), keypad.c/keypad.h (teclado por ADC), uart.c/uart.h (comunicação serial), adxl345.c/adxl345.h (acelerômetro por I2C) e main.c (aplicação). O sistema opera com o oscilador interno HSI de 16 MHz, sem configuração de PLL, e os valores de prescaler e de baud rate foram calculados para essa frequência. Assim como na AV1, o Manual de Referência RM0383 (família STM32F411) foi usado como referência de registradores, pois GPIO, RCC, ADC, USART e TIM2 pertencem ao mesmo bloco de propriedade intelectual do STM32F446. O código-fonte completo dos módulos está no Apêndice A.

4.2 Hardware e conexões

O shield LCD Keypad, no formato Arduino, foi encaixado diretamente nos conectores Arduino da NUCLEO, sem fios nem jumpers. O shield possui 16 pinos no conector digital e 12 nos conectores de energia e analógico, contra 18 e 14 soquetes na placa; o encaixe é feito alinhando as extremidades dos pinos D0 e A0, de modo que ficam sem uso, junto ao conector USB, os soquetes SCL/D15 e SDA/D14 (conector digital) e NC e IOREF (conector de energia). O sinal RW do LCD não é controlado pelo microcontrolador: o display é apenas escrito e os tempos são garantidos por atrasos, sem leitura do busy flag. A Tabela 1 resume as ligações utilizadas.

Sinal do LCD	Pino Arduino	Pino STM32	Função
RS	D8	PA9	0 = comando, 1 = dado
E	D9	PC7	Pulso de habilitação
D4	D4	PB5	Bit 0 do nibble
D5	D5	PB4	Bit 1 do nibble
D6	D6	PB10	Bit 2 do nibble
D7	D7	PA8	Bit 3 do nibble
Teclas	A0	PA0	Divisor resistivo (ADC1, canal 0)

4.3 Driver do LCD (Parte 1)

A função lcd_gpio_init() habilita o clock de GPIOA, GPIOB e GPIOC e configura os seis pinos como saídas push-pull, de baixa velocidade e sem resistores de pull-up ou pull-down. A função lcd_set_data4() copia os quatro bits de um nibble para PA8 (D7), PB10 (D6), PB4 (D5) e PB5 (D4), alterando apenas esses bits do registrador ODR. A função interna lcd_write_nibble(nibble, rs) define o nível de RS, coloca o nibble nos pinos de dados, sobe o sinal E, aguarda 1 µs, desce E (instante em que o LCD captura o dado) e

aguarda 50 µs para o processamento. As funções lcd_send_cmd() e lcd_send_data() enviam um byte em dois nibbles, o mais significativo primeiro, com RS = 0 e RS = 1, respectivamente.

A função lcd_init() segue a sequência da Tabela 2. Os passos 2 a 5 enviam nibbles isolados, pois o display ainda pode estar em modo de 8 bits e só enxerga as linhas D4 a D7; a partir do passo 6, já em modo de 4 bits, cada comando é enviado em dois nibbles.

Passo	Envio	Espera depois	Função
1	—	50 ms	Estabilização após energizar
2	nibble 0x3	5 ms	Function set (8 bits), 1ª vez
3	nibble 0x3	150 µs	Function set (8 bits), 2ª vez
4	nibble 0x3	150 µs	3ª vez: estado conhecido
5	nibble 0x2	150 µs	Passa para o modo de 4 bits
6	comando 0x28	50 µs por nibble	4 bits, 2 linhas, fonte 5×8
7	comando 0x08	50 µs por nibble	Display desligado
8	comando 0x01	2 ms	Limpa o display
9	comando 0x06	50 µs por nibble	Cursor incrementa, sem deslocar a tela
10	comando 0x0C	50 µs por nibble	Display ligado, sem cursor nem blink

A função lcd_clear() envia o comando 0x01 e espera 2 ms, pois essa instrução leva até 1,52 ms [4]. A função lcd_put_cur(linha, coluna) envia o comando Set DDRAM Address, cujo código é 0x80 | coluna para a linha 0 e 0xC0 | coluna para a linha 1 (a segunda linha começa no endereço 0x40 da DDRAM). A função lcd_send_string() percorre a string até o caractere nulo e envia cada caractere com lcd_send_data().

4.4 Base de tempo (Parte 3)

O TIM2 é configurado uma única vez em tim2_init(): prescaler PSC = 15 (divisão por 16, ou seja, um passo por microssegundo com o clock de 16 MHz), ARR = 0xFFFFFFFF e contador habilitado, em contagem livre e sem interrupção. A função delay_us(us) lê o contador no início e aguarda até que a diferença CNT − início atinja us. Como a subtração é feita com inteiros sem sinal, o resultado continua correto mesmo que o contador dê a volta. A função delay_ms(ms) repete delay_us(1000) por ms vezes. A resolução é de 1 µs e o atraso real fica entre us e us + 1 microssegundos, pois a leitura inicial do contador cai no meio de um passo.

4.5 Leitura e decodificação das teclas (Parte 2)

O sinal do teclado chega ao pino A0 do shield, que, na NUCLEO, é o PA0, ligado ao canal 0 do ADC1. O enunciado menciona o canal 1 (A1); porém, a ligação física do shield encaixado cai em A0, e o modelo de solução fornecido também configura o canal 0. O canal foi parametrizado por uma constante (KEYPAD_CH) para facilitar a alteração.

A função keypad_init() habilita os clocks de GPIOA e ADC1, coloca PA0 em modo analógico, seleciona o canal 0 como única conversão da sequência (ADC_SQR3 e ADC_SQR1), configura o tempo de amostragem de 480 ciclos (ADC_SMPR2), pois a rede resistiva tem impedância alta, e liga o ADC (bit

ADON). A função keypad_read_raw() dispara a conversão por software (SWSTART), aguarda a flag EOC e retorna o valor de 12 bits de ADC1->DR. A função keypad_read_key() compara esse valor com faixas e devolve o código da tecla. As faixas foram definidas a partir de medições na placa (Seção 5.2), no ponto médio entre os valores de teclas vizinhas, conforme a Tabela 3.

Tecla	Valor medido (12 bits)	Faixa decodificada
RIGHT	≈ 1	< 300
UP	≈ 570	300 a 1019
DOWN	≈ 1471	1020 a 1899
LEFT	≈ 2332	1900 a 2999
SELECT	≈ 3679	3000 a 3889
Nenhuma tecla	4095	≥ 3890

4.6 Comunicação serial

A função uart2_init() coloca PA2 e PA3 em função alternativa AF7 (USART2), habilita o clock da USART2 (barramento APB1) e configura o baud rate em 115200 bps (BRR = 16 MHz / 115200, arredondado para 139), com 8 bits de dados, sem paridade e 1 bit de parada, e transmissor e receptor habilitados. Como o laço principal também precisa ler o teclado, a recepção é feita por consulta à flag RXNE, sem bloqueio: uart2_rx_ready() informa se há um byte pendente e uart2_read() o lê (a leitura de USART_DR limpa a flag). Os dados chegam ao computador pela porta serial virtual do ST-LINK, observada com um programa de terminal configurado para 115200 bps. A transmissão, usada para responder ao comando serial, é feita por uart2_write_char(), que aguarda a flag TXE (registrador de transmissão vazio) e escreve em USART_DR, e pelas funções uart2_write_string() e uart2_write_int(), esta última convertendo um inteiro para decimal sem uso de printf.

4.7 Aplicação principal (Parte 4)

A função main() inicializa o GPIO do LCD, o TIM2, o LCD, o teclado e a USART2. Em seguida, mostra na linha 1 o texto SIST. TEMPO REAL e na linha 2 o texto * U N I F O R *, aguarda 3 s com delay_ms(3000), limpa o display e exibe TECLA ACIONADA: na linha 1 e NONE na linha 2.

No laço principal, a cada 10 ms, ocorrem duas verificações. Na primeira, a tecla é lida e só é aceita quando duas leituras consecutivas coincidem (um debounce simples); quando a tecla muda, seu nome é reescrito na linha 2 e, ao soltar a tecla, a tela volta para NONE. Os nomes têm 6 caracteres, preenchidos com espaços, para apagar o nome anterior. Na segunda, se há um byte recebido e ele é imprimível, é escrito na linha 2, coluna 7 (índice 6 em lcd_put_cur()), sem interferir no nome da tecla, que ocupa as colunas 1 a 6; caracteres não imprimíveis, como o Enter, são ignorados.

Na mesma verificação da serial, quando o byte recebido é o caractere A (maiúsculo ou minúsculo), a função accel_report() é chamada: ela lê os três eixos do ADXL345 e envia pela serial uma linha no formato X=<bruto> (<mg> mg) Y=<bruto> (<mg> mg) Z=<bruto> (<mg> mg). Se o sensor não respondeu na inicialização (por exemplo, por fio solto), a inicialização é tentada de novo a cada comando e, em caso de falha, é enviada uma mensagem de erro, sem travar o laço principal.

4.8 Acelerômetro ADXL345 e comando serial "A"

O módulo ADXL345 (placa de breakout) foi ligado à NUCLEO por quatro fios, sem encaixe no shield: SCL e SDA nos pinos PB8 (D15) e PB9 (D14), que o shield deixa livres, e a alimentação de 3,3 V e o GND da própria NUCLEO. O módulo mantém o pino CS em nível alto, o que seleciona o modo I²C. A Tabela 4 resume as ligações.

Sinal do módulo	Pino da NUCLEO	Função
SCL	PB8 (D15)	Relógio do I²C (I2C1_SCL, AF4)
SDA	PB9 (D14)	Dados do I²C (I2C1_SDA, AF4)
VCC	3V3	Alimentação de 3,3 V
GND	GND	Referência de terra
CS	Nível alto	Seleciona o modo I²C

A função adxl345_init() configura PB8 e PB9 em função alternativa AF4 (I2C1), como open-drain e com pull-up interno de apoio (o módulo já possui seus resistores), habilita o clock do I2C1 (APB1) e programa o periférico para 100 kHz: I2C_CR2.FREQ = 16 (clock do APB1, em MHz), I2C_CCR = 80 (meio período de 5 µs sobre o período de 62,5 ns do clock) e I2C_TRISE = 17. Em seguida, lê o registrador DEVID nos dois endereços possíveis (0x53 e 0x1D) e só aceita o sensor se o valor lido for 0xE5. Por fim, escreve BW_RATE = 0x0A (100 Hz), DATA_FORMAT = 0x08 (resolução total, faixa de ±2 g, 3,9 mg por LSB) e POWER_CTL = 0x08 (bit Measure), conforme a Tabela 5.

Registrador	Endereço	Valor	Efeito
DEVID	0x00	leitura	Deve retornar 0xE5 (identifica o ADXL345)
BW_RATE	0x2C	0x0A	Taxa de saída de dados de 100 Hz
DATA_FORMAT	0x31	0x08	FULL_RES, faixa de ±2 g (3,9 mg/LSB)
POWER_CTL	0x2D	0x08	Measure = 1: inicia as medições
DATAX0 a DATAZ1	0x32 a 0x37	leitura	Seis bytes: X, Y e Z, byte baixo primeiro

A leitura dos eixos é feita por adxl_read_regs(), que escreve o endereço do registrador DATAX0 (mestre-transmissor), gera um START repetido em modo de leitura e lê seis bytes seguindo a sequência do manual de referência para mais de dois bytes: ACK desligado antes dos três últimos bytes, STOP e leitura do último byte pela flag RXNE [1]. Cada eixo é formado por dois bytes em complemento de dois, byte baixo primeiro, e é convertido para mg multiplicando por 39 e dividindo por 10 (3,9 mg por LSB). Todas as esperas por flags do I2C têm tempo máximo de 5 ms, medido pelo TIM2, e o NACK é detectado pela flag AF; assim, a ausência do sensor resulta em um código de erro, e não no travamento do programa.

5. Resultados e Discussão

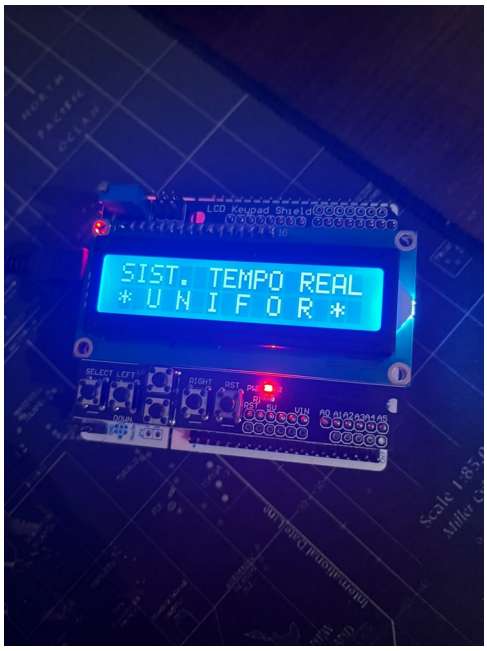


*Figura 1 — Montagem física: shield LCD Keypad encaixado na NUCLEO-STM32F446RE, exibindo a tela de boas-vindas (SIST. TEMPO REAL / * U N I F O R *) após o reset.*

5.1 Validação das telas

Após a gravação do firmware, o display apresentou a tela de boas-vindas nas duas linhas (Figura 1), mantida por 3 s conforme delay_ms(3000), seguida da tela de espera de tecla, com TECLA ACIONADA: na linha 1 e NONE na linha 2 (Figura 2). A correta exibição das duas linhas confirma o driver do LCD: a inicialização em modo de 4 bits, o envio de dados e comandos, o posicionamento do cursor e a limpeza do display.



Figura 2 — Tela de espera de tecla: TECLA ACIONADA: com nenhuma tecla pressionada (NONE) e o caractere r, enviado pelo terminal serial, na coluna 7 da linha 2.

5.2 Calibração do teclado: problema real de bancada

Na primeira versão, os limites das faixas foram estimados a partir do esquema do shield, que indica as tensões de cada tecla (valores de 10 bits com referência de 5 V), convertidos para 12 bits e referência de 3,3 V. Ao gravar o firmware, as teclas foram decodificadas de forma errada: LEFT aparecia como DOWN, DOWN aparecia como UP e SELECT aparecia como NONE.

Para diagnosticar, o firmware foi substituído temporariamente por um teste que mostrava, na linha 1, o valor cru do ADC (RAW) e, na linha 2, a tecla decodificada. Sem nenhuma tecla pressionada, o valor foi 4095, pois a entrada fica em 5 V, acima da referência do ADC, e satura. Com cada tecla mantida, foram obtidos os valores da Tabela 3, com variação de poucas contagens. As leituras se mostraram bem separadas, inclusive SELECT (≈ 3679) e repouso (4095), o que permitiu definir os limites no ponto médio entre valores vizinhos. Com os novos limites, as cinco teclas passaram a ser identificadas corretamente (Figura 3 mostra a tecla UP). A lição é que as tensões teóricas do esquema não substituem a medição na placa: a tolerância dos resistores e a diferença de referência alteram os valores.

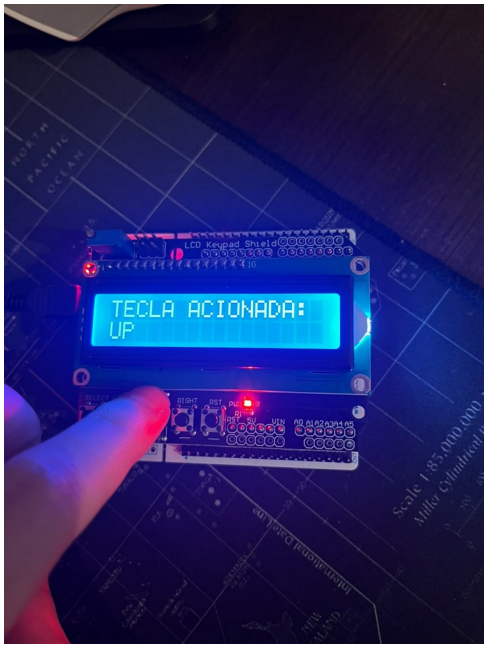


Figura 3 — Tecla UP pressionada: a linha 2 passa de NONE para UP enquanto a tecla é mantida; ao soltá-la, o display volta para NONE.

5.3 Validação da recepção serial

Ao digitar um caractere no terminal serial a 115200 bps, ele foi exibido na linha 2, coluna 7 (Figura 2, caractere r). O caractere permanece no display quando a tecla pressionada muda, pois o nome da tecla e o caractere ocupam regiões diferentes da linha, e o laço continua lendo as teclas enquanto aguarda a serial, o que comprova que a recepção por polling não bloqueia o sistema. Ao reiniciar a placa, o display é reinicializado e o caractere deixa de ser exibido até que um novo seja recebido, já que o programa não armazena o último valor.

5.4 Dificuldade de projeto: atraso curto com o TIM2

Na revisão do código, antes da gravação, constatou-se que a primeira versão de delay_us() usava ARR = 0xFFFFFFFF e esperava a flag de estouro (UIF) a cada microssegundo. Como o TIM2 tem 32 bits, a flag só seria levantada após 2^32 µs (cerca de 71 minutos), e a primeira chamada de delay_us() travaria o programa. O teste inicial com o LED da placa, que usava apenas delay_ms() (ARR = 999), não revelaria o erro. A correção foi manter o timer em contagem livre e medir o tempo pela diferença entre leituras de CNT (Seção 4.4), o que também elimina a reconfiguração do timer a cada chamada.

5.5 Leitura do acelerômetro pelo comando serial "A"

O ADXL345 foi ligado à NUCLEO por quatro fios (Figura 4). Como o shield do LCD cobre os conectores da placa e não havia espaço físico para manter as duas montagens ao mesmo tempo, o item 1 (shield, teclado e serial) e o item 2 (acelerômetro) foram demonstrados em montagens separadas, com o mesmo firmware gravado: sem o sensor, o comando A responde apenas com a mensagem de erro, e o restante do sistema continua funcionando.

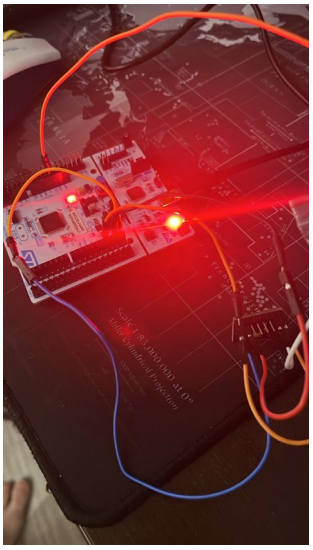


Figura 4 — Montagem do teste do ADXL345: módulo ligado por jumpers à NUCLEO-STM32F446RE (SCL, SDA, alimentação e GND), com a placa alimentada pela porta USB.

Com o terminal serial a 115200 bps, a cada caractere A digitado a placa respondeu com uma linha contendo os três eixos (Figura 5), cada um em valor bruto e em mg. Na primeira posição, o valor de Z ficou próximo de +1 g (924 mg), com X e Y menores; ao girar a placa, a gravidade passou a se concentrar no eixo Y (−1017 mg), e Z caiu para cerca de 0,2 g. A Tabela 6 resume duas linhas da Figura 5, com o módulo do vetor calculado por $\sqrt{(X^2 + Y^2 + Z^2)}$.

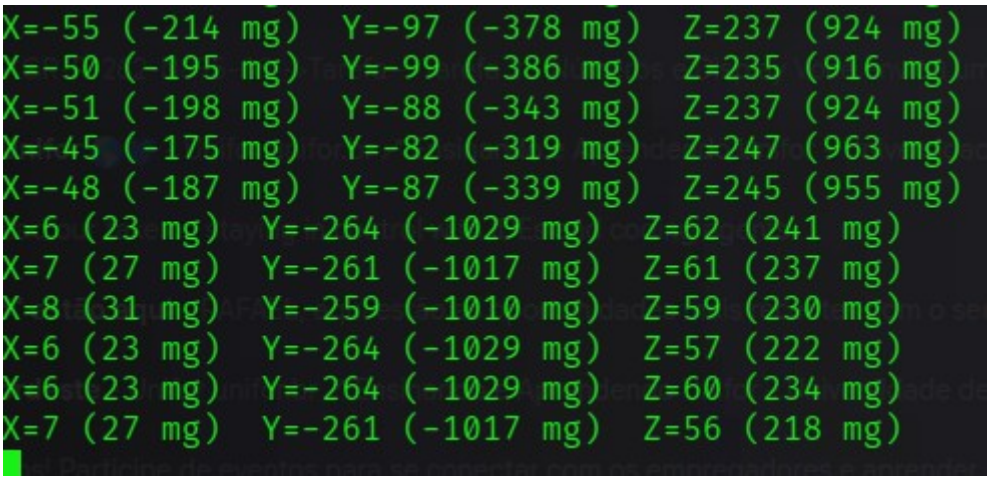


Figura 5 — Resposta serial ao comando A, exibida no terminal: valores brutos e em mg dos eixos X, Y e Z em duas posições da placa.

Posição	X (mg)	Y (mg)	Z (mg)	Módulo (mg)
1	−214	−378	924	1021
2	27	−1017	218	1040

Nas duas posições, o módulo do vetor resultou em valor próximo de 1 g (1021 mg e 1040 mg), o que confirma que os três eixos estão sendo lidos e escalados corretamente: a gravidade apenas se redistribui entre os eixos quando a orientação muda. A diferença em relação a 1000 mg (de 2 a 4 %) é compatível com os erros de offset e de sensibilidade do sensor, que não foi calibrado neste trabalho [5].

6. Conclusão

A atividade permitiu implementar, com acesso direto aos registradores, um sistema completo de interface com o operador na placa NUCLEO-STM32F446RE: o driver do LCD 16x2 em modo de 4 bits, a leitura do teclado por ADC, a base de tempo por temporizador, a recepção serial e a leitura de um acelerômetro ADXL345 por I²C. Os testes em bancada confirmaram os requisitos: a tela de boas-vindas, o atraso de 3 s, a tela de espera, o nome da tecla enquanto ela é pressionada, o retorno a NONE ao soltar e o caractere recebido pela serial na linha 2, coluna 7. Para o acelerômetro, o comando A retornou os três eixos, com módulo do vetor próximo de 1 g nas posições testadas.

Os dois problemas encontrados reforçam práticas de desenvolvimento embarcado: calibrar a decodificação do teclado a partir de medições reais, em vez de valores teóricos, e revisar as condições de espera de cada rotina de atraso, já que um erro na configuração do temporizador pode travar o sistema de forma silenciosa. A sequência de inicialização do HD44780, com nibbles isolados antes da mudança para 4 bits, também mostrou a importância de seguir as temporizações do fabricante.

Como trabalho futuro, propõe-se receber a serial por interrupção (RXNEIE), liberando o laço principal da consulta periódica, implementar o debounce com o próprio temporizador e substituir as esperas fixas do LCD pela leitura do busy flag, além de calibrar o offset do ADXL345 e usar seus pinos de interrupção (INT1 e INT2) para sinalizar dados prontos sem consulta.

Referências

[1] STMICROELECTRONICS. RM0383 — Reference manual: STM32F411xC/E advanced Arm® -based 32-bit MCUs. Rev. 3, nov. 2018.

[2] HITACHI. HD44780U (LCD-II) — Dot Matrix Liquid Crystal Display Controller/Driver. Folha de dados.

[3] STMICROELECTRONICS. UM1724 — User manual: STM32 Nucleo-64 boards (MB1136). DocID025833 Rev 5, jun. 2014.

[4] IMBIRIBA. Projeto 2 AV2 — Uso do LCD Shield 16x2 com teclado e serial (enunciado da atividade). Disciplina Sistema de Tempo Real (STR). Universidade de Fortaleza (UNIFOR), 2026.

[5] ANALOG DEVICES. ADXL345 — 3-Axis, ±2 g/±4 g/±8 g/±16 g Digital Accelerometer. Folha de dados.

Apêndice A — Códigos-fonte

Listagem completa dos arquivos do projeto av2_shield_lcd_serial (pastas Inc e Src).

A.1 Inc/lcd.h

```
#ifndef LCD_H
#define LCD_H

#include <stdint.h>
#include "stm32f4xx.h"


void lcd_gpio_init(void);
void lcd_set_data4(uint8_t);
void lcd_send_cmd(uint8_t);
void lcd_send_data(uint8_t);
void lcd_init(void);
void lcd_clear(void);
void lcd_put_cur(int, int);
void lcd_send_string(const char *);

#endif
```

A.2 Src/lcd.c

```
#include "lcd.h"
#include "tim.h"

static void lcd_write_nibble(uint8_t nibble, uint8_t rs);

void lcd_gpio_init(void)
{
    /* PA8 = D7; PA9 = RS. */
    const uint32_t pins_a = (1U << 8U) | (1U << 9U);
    const uint32_t mask_a = (3U << 16U) | (3U << 18U);

    /* PB4 = D5; PB5 = D4; PB10 = D6. */
    const uint32_t pins_b = (1U << 4U) | (1U << 5U) | (1U << 10U);
    const uint32_t mask_b = (3U << 8U) | (3U << 10U) | (3U << 20U);

    /* PC7 = E. */
    const uint32_t pins_c = (1U << 7U);
    const uint32_t mask_c = (3U << 14U);

    // AHB1ENR = registrador do módulo RCC que controla os clocks.
    /* Habilita o clock das tres portas utilizadas pelo LCD. */
    RCC->AHB1ENR = RCC->AHB1ENR | RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOBEN | RCC_AHB1ENR_GPIOCEN;

    // Forma parar gerar um pequeno delay antes de acessarmos os GPIO para dar tempo de eles serem
habilitados certinho.
    // basicamente só busca um valor no registrador e não retorna nada.
    (void)RCC->AHB1ENR;

    // BSRR = Bit Set/Reset Register. É um registrador do GPIO que muda o nível de saída de pinos
individuais, de forma atômica
    // do bit 0 a 15 é set, aonde se eu colocar o bit em 1, ele estará setado, do bit 16 a 31, é o
reset, se o bit for selecionado lá, ele vai
    // para 0, se os 2 forem acionados, ele fica em set.
    // O << 16U jogs os bits para a metade de reset então todos os pinos vão a 0, garantido que D4-D7,
RS e E comecem em nível baixo, só para
    // garantir que caso E de um pulso, não vá sair lixo no LCD..
    GPIOA->BSRR = pins_a << 16U; // pins_a = PA8 e PA9
    GPIOB->BSRR = pins_b << 16U; // PB4, PB5, PB10
    GPIOC->BSRR = pins_c << 16U; // PC7

    /* Saidas push-pull: o STM32 produz tanto nivel alto como baixo. */
    // OTYPER = Output Type Register. Define, para cada pino configurado como saída, como o circuito de
saída é montado.
    // O objetivo aqui é só deixar os pinos mencionados em push-pull (que é o estado 0 nesse caso).
    GPIOA->OTYPER &= ~pins_a;
    GPIOB->OTYPER &= ~pins_b;
    GPIOC->OTYPER &= ~pins_c;
```

```
    /* Baixa velocidade de comutacao: suficiente para esta interface. */
    // OSPEEDR = Output Speed Register. Define a velocidade de comutação (slew rate) do driver de saída
    // de cada pino: o quão rápido o pino sobe de 0 para 1 e desce de 1 para 0.
    GPIOA->OSPEEDR &= ~mask_a;
    GPIOB->OSPEEDR &= ~mask_b;
    GPIOC->OSPEEDR &= ~mask_c;

    /* Sem resistores internos de pull-up ou pull-down. */
    // PUPDR = Pull-Up/Pull-Down Register. Liga um resistor interno (de aproximadamente 40 kΩ)
    // entre o pino e o VDD (pull-up) ou o GND (pull-down), ou nenhum.
    GPIOA->PUPDR &= ~mask_a;
    GPIOB->PUPDR &= ~mask_b;
    GPIOC->PUPDR &= ~mask_c;

    /* MODER: cada pino ocupa dois bits; 01 significa saida. */
    // define se o pino é entrada, saída, analógico ou função alternativa, nesse caso, todos são saida.
    GPIOA->MODER = (GPIOA->MODER & ~mask_a) | (1U << 16U) | (1U << 18U);
    GPIOB->MODER = (GPIOB->MODER & ~mask_b) | (1U << 8U) | (1U << 10U) | (1U << 20U);
    GPIOC->MODER = (GPIOC->MODER & ~mask_c) | (1U << 14U);
}

void lcd_set_data4(uint8_t value)
{
    /* Bit 3 do grupo -> D7 do LCD -> PA8 da placa STM32. */
    if (value & (1U << 3U))
        GPIOA->ODR |= (1U << 8U);
    else
        GPIOA->ODR &= ~(1U << 8U);

    /* Bit 2 do grupo -> D6 do LCD -> PB10 da placa STM32. */
    if (value & (1U << 2U))
        GPIOB->ODR |= (1U << 10U);
    else
        GPIOB->ODR &= ~(1U << 10U);

    /* Bit 1 do grupo -> D5 do LCD -> PB4 da placa STM32. */
    if (value & (1U << 1U))
        GPIOB->ODR |= (1U << 4U);
    else
        GPIOB->ODR &= ~(1U << 4U);

    /* Bit 0 do grupo -> D4 do LCD -> PB5 da placa STM32. */
    if (value & (1U << 0U))
        GPIOB->ODR |= (1U << 5U);
    else
        GPIOB->ODR &= ~(1U << 5U);
}

static void lcd_write_nibble(uint8_t nibble, uint8_t rs){
    // Controla se será um comando (0) ou envio de dado (1)
    if (rs)
        GPIOA->ODR |= (1U << 9U); // Coloca o bit 9 (PA9) em 1
    else
        GPIOA->ODR &= ~(1U << 9U); // Coloca o bit 9 (PA9) em 0

    lcd_set_data4(nibble); // Pulso em E (PC7)

    GPIOC->ODR |= (1U << 7U);     /* E = 1 */
    delay_us(1);
    GPIOC->ODR &= ~(1U << 7U);    /* E = 0: o LCD captura na borda de descida */
    delay_us(50);                 /* tempo para o LCD processar */

}

void lcd_send_cmd(uint8_t cmd)
{
    lcd_write_nibble(cmd >> 4, 0U);      /* nibble alto */
    lcd_write_nibble(cmd & 0x0FU, 0U);   /* nibble baixo */
}

void lcd_send_data(uint8_t data)
{
    lcd_write_nibble(data >> 4, 1U);
    lcd_write_nibble(data & 0x0FU, 1U);
```

```
}

void lcd_init(void)
{
    /* Espera a tensao e o LCD estabilizarem apos ligar. */
    delay_ms(50U);

    /* O LCD pode estar em modo 8 bits: so D4-D7 estao ligados, entao
     * enviamos meio byte (nibble 0x3) tres vezes para leva-lo a um estado conhecido. */
    lcd_write_nibble(0x3U, 0U);
    delay_ms(5U);               /* > 4,1 ms */
    lcd_write_nibble(0x3U, 0U);
    delay_us(150U);             /* > 100 us */
    lcd_write_nibble(0x3U, 0U);
    delay_us(150U);

    /* Nibble 0x2: passa o LCD para o modo 4 bits. */
    lcd_write_nibble(0x2U, 0U);
    delay_us(150U);

    /* A partir daqui ja em 4 bits: cada comando sao dois nibbles. */
    lcd_send_cmd(0x28U);        /* Function set: 4 bits, 2 linhas, fonte 5x8 */
    lcd_send_cmd(0x08U);        /* Display off */
    lcd_send_cmd(0x01U);        /* Clear display */
    delay_ms(2U);               /* Clear demora ate 1,52 ms */
    lcd_send_cmd(0x06U);        /* Entry mode: cursor incrementa, sem deslocar a tela */
    lcd_send_cmd(0x0CU);        /* Display on, cursor off, blink off */
}

void lcd_clear(void)
{
    lcd_send_cmd(0x01U);        /* Clear display */
    delay_ms(2U);               /* Clear demora ate 1,52 ms */
}

// Posiciona cursor do LCD, escolhe em qual linha e coluna o próximo caractere será escrito.
void lcd_put_cur(int row, int col)
{
    /* Set DDRAM address: bit 7 em 1 + endereco. Linha 0 comeca em 0x00, linha 1 em 0x40. */
    uint8_t addr = (uint8_t)col & 0x3FU;

    if (row == 0)
        addr |= 0x80U;
    else
        addr |= 0xC0U;

    lcd_send_cmd(addr);
}

void lcd_send_string(const char *str)
{
    while (*str)
        lcd_send_data((uint8_t)*str++);
}
```

A.3 Inc/tim.h

```
#ifndef TIM_H
#define TIM_H

#include <stdint.h>
#include "stm32f4xx.h"

void tim2_init(void);
void delay_us(uint32_t);
void delay_ms(uint32_t);

#endif
```

A.4 Src/tim.c

```
#include "tim.h"

void tim2_init(void)
```

```
{
    /* Habilita o acesso ao timer interno da placa STM32. */
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
    (void)RCC->APB1ENR;

    //CR1 = Control Register 1 do timer. É o registrador principal de controle do TIM2.
    TIM2->CR1 = 0U;   /* Deixa o contador parado durante a configuracao. */

    // DIER (DMA/Interrupt Enable Register): habilita as interrupções e as requisições de DMA do timer.
    TIM2->DIER = 0U;  /* Sem interrupcoes. */

    // PSC (Prescaler): divide o clock de entrada por PSC + 1. Com PSC = 15, 16 MHz ÷ 16 = 1 MHz, ou
seja, 1 contagem por µs.
    TIM2->PSC = 16U - 1U;      /* Divide por 16: um passo a cada 1 microssegundo. */

    // ARR (Auto-Reload Register): valor máximo da contagem. Ao chegar nele, o contador volta a 0.
    // Com 0xFFFFFFFF (32 bits), só dá a volta depois de ~71 min.
    TIM2->ARR = 0xFFFFFFFFU;   /* TIM2 tem 32 bits: so da a volta apos ~71 minutos. */

    // EGR (Event Generation Register): permite gerar eventos por software.
    TIM2->EGR = TIM_EGR_UG;    /* Aplica a configuracao e reinicia a contagem. */

    // SR (Status Register): guarda os flags do timer, no caso, limpamos ele.
    TIM2->SR = 0U;
    TIM2->CR1 |= TIM_CR1_CEN;  /* Contador livre: nunca mais para, CNT sobe 1 por us. */
}


void delay_us(uint32_t us)
{
    uint32_t start = TIM2->CNT;

    /* A subtracao sem sinal continua correta mesmo se CNT der a volta. */
    while ((TIM2->CNT - start) < us) {}
}


void delay_ms(uint32_t ms)
{
    while (ms > 0U)
    {
        delay_us(1000U);
        ms--;
    }
}
```

A.5 Inc/keypad.h

```
#ifndef KEYPAD_H
#define KEYPAD_H

#include <stdint.h>
#include "stm32f4xx.h"

#define KEY_RIGHT   0
#define KEY_UP      1
#define KEY_DOWN    2
#define KEY_LEFT    3
#define KEY_SELECT  4
#define KEY_NONE    5

void keypad_init(void);
uint16_t keypad_read_raw(void);
uint8_t keypad_read_key(void);

#endif
```

A.6 Src/keypad.c

```
#include "keypad.h"

/* A0 do shield cai em A0 da Nucleo = PA0 = ADC1_IN0. Se a medicao indicar PA1, troque para 1. */
#define KEYPAD_CH   0U
```

```
/* Limites no meio entre as leituras medidas na placa (12 bits):
 * RIGHT ~1, UP ~570, DOWN ~1471, LEFT ~2332, SELECT ~3679, sem tecla 4095. */
#define TH_RIGHT    300U
#define TH_UP       1020U
#define TH_DOWN     1900U
#define TH_LEFT     3000U
#define TH_SELECT   3890U

void keypad_init(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;
    (void)RCC->APB2ENR;

    /* MODER: 11 = analogico no pino do canal escolhido. */
    GPIOA->MODER |= (3U << (2U * KEYPAD_CH));

    /* Tempo de amostragem de 480 ciclos: a rede resistiva do shield tem impedancia alta. */
    ADC1->SMPR2 |= (7U << (3U * KEYPAD_CH));

    ADC1->SQR1 = 0U;           /* Sequencia de 1 conversao. */
    ADC1->SQR3 = KEYPAD_CH;    /* A primeira (e unica) conversao e o canal do teclado. */
    ADC1->CR2 |= ADC_CR2_ADON; /* Liga o ADC. */
}

uint16_t keypad_read_raw(void)
{
    ADC1->CR2 |= ADC_CR2_SWSTART;            /* Inicia uma conversao. */
    while ((ADC1->SR & ADC_SR_EOC) == 0U) {} /* Espera o fim da conversao. */
    return (uint16_t)ADC1->DR;               /* A leitura limpa o EOC. */
}

uint8_t keypad_read_key(void)
{
    uint16_t v = keypad_read_raw();

    if (v < TH_RIGHT) return KEY_RIGHT;
    if (v < TH_UP)    return KEY_UP;
    if (v < TH_DOWN)  return KEY_DOWN;
    if (v < TH_LEFT)  return KEY_LEFT;
    if (v < TH_SELECT) return KEY_SELECT;
    return KEY_NONE;
}
```

A.7 Inc/uart.h

```
#ifndef UART_H
#define UART_H

#include <stdint.h>
#include "stm32f4xx.h"

void uart2_init(void);
uint8_t uart2_rx_ready(void);
char uart2_read(void);
void uart2_write_char(char);
void uart2_write_string(const char *);
void uart2_write_int(int32_t);

#endif
```

A.8 Src/uart.c

```
#include "uart.h"

#define SYS_FREQ        16000000U   /* HSI de 16 MHz (padrao apos o reset); APB1 sem prescaler */
#define UART_BAUDRATE   115200U

/* Configura PA2 (TX) e PA3 (RX) como USART2 (AF7) a 115200 bps, 8N1. */
void uart2_init(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
    (void)RCC->APB1ENR;
```

```
    /* PA2 e PA3 em modo funcao alternativa (MODER = 10). */
    GPIOA->MODER = (GPIOA->MODER & ~((3U << 4U) | (3U << 6U)))
                 | (2U << 4U) | (2U << 6U);

    /* AF7 = USART2 em AFR[0] nos bits 11:8 (PA2) e 15:12 (PA3). */
    GPIOA->AFR[0] = (GPIOA->AFR[0] & ~(0xFFU << 8U)) | (0x77U << 8U);

    /* BRR = clock / baudrate (oversampling de 16), arredondado: 16 MHz / 115200 = 139. */
    USART2->BRR = (SYS_FREQ + (UART_BAUDRATE / 2U)) / UART_BAUDRATE;

    USART2->CR1 = USART_CR1_TE | USART_CR1_RE;
    USART2->CR1 |= USART_CR1_UE;
}

/* Retorna 1 se ha um byte recebido esperando leitura (nao bloqueia). */
uint8_t uart2_rx_ready(void)
{
    return (USART2->SR & USART_SR_RXNE) != 0U;
}

/* Le o byte recebido; a leitura do DR limpa o RXNE. Use depois de uart2_rx_ready(). */
char uart2_read(void)
{
    return (char)USART2->DR;
}

/* Envia um caractere: espera o TXE (registrador de transmissao vazio) e escreve no DR. */
void uart2_write_char(char c)
{
    while ((USART2->SR & USART_SR_TXE) == 0U) {}
    USART2->DR = (uint8_t)c;
}

void uart2_write_string(const char *str)
{
    while (*str)
        uart2_write_char(*str++);
}

/* Envia um inteiro em decimal, sem printf. */
void uart2_write_int(int32_t value)
{
    char digits[11];
    uint8_t n = 0U;
    uint32_t v;

    if (value < 0)
    {
        uart2_write_char('-');
        v = (uint32_t)(-(value + 1)) + 1U;   /* Evita overflow em INT32_MIN. */
    }
    else
    {
        v = (uint32_t)value;
    }

    /* Os digitos saem do menos para o mais significativo, por isso ficam guardados e sao enviados ao
contrario. */
    do
    {
        digits[n++] = (char)('0' + (v % 10U));
        v /= 10U;
    } while (v > 0U);

    while (n > 0U)
        uart2_write_char(digits[--n]);
}
```

A.9 Inc/adxl345.h

```
#ifndef ADXL345_H
#define ADXL345_H

#include <stdint.h>
```

```
#include "stm32f4xx.h"

#define ADXL345_OK    0U
#define ADXL345_ERR   1U

/* Configura o I2C1 (PB8 = SCL, PB9 = SDA) e liga o ADXL345. Retorna ADXL345_OK se o sensor respondeu.
*/
uint8_t adxl345_init(void);

/* Le os tres eixos (valores brutos, 3,9 mg por LSB). Retorna ADXL345_OK se a leitura deu certo. */
uint8_t adxl345_read_xyz(int16_t *x, int16_t *y, int16_t *z);

#endif
```

A.10 Src/adxl345.c

```
#include "adxl345.h"
#include "tim.h"

/* Endereco de 7 bits: 0x53 com o pino SDO em GND, 0x1D com SDO em VCC. */
#define ADXL_ADDR_LOW     0x53U
#define ADXL_ADDR_HIGH    0x1DU

#define REG_DEVID         0x00U   /* Sempre le 0xE5. */
#define REG_BW_RATE       0x2CU
#define REG_POWER_CTL     0x2DU
#define REG_DATA_FORMAT   0x31U
#define REG_DATAX0        0x32U   /* X0 X1 Y0 Y1 Z0 Z1: 6 bytes seguidos, byte baixo primeiro. */

#define DEVID_VALUE       0xE5U

/* Tempo maximo de espera por qualquer evento do I2C: sem sensor, o programa nao trava. */
#define I2C_TIMEOUT_US    5000U

static uint8_t adxl_addr = ADXL_ADDR_LOW;

/* Reconfigura o periferico I2C1 do zero (tambem serve para destravar o barramento). */
static void i2c1_config(void)
{
    I2C1->CR1 = I2C_CR1_SWRST;     /* Reset do periferico. */
    I2C1->CR1 = 0U;

    I2C1->CR2 = 16U;               /* FREQ = 16 MHz (clock do APB1). */
    I2C1->CCR = 80U;               /* 100 kHz: meio periodo = 80 x 62,5 ns = 5 us. */
    I2C1->TRISE = 17U;             /* 1000 ns / 62,5 ns + 1. */
    I2C1->CR1 |= I2C_CR1_PE;
}

static void i2c1_init(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
    RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;
    (void)RCC->APB1ENR;

    /* PB8 e PB9: open-drain (o I2C so puxa para 0, o resistor leva a 1). */
    GPIOB->OTYPER |= (1U << 8U) | (1U << 9U);

    /* Pull-up interno de apoio (o modulo ja costuma ter os seus). */
    GPIOB->PUPDR = (GPIOB->PUPDR & ~((3U << 16U) | (3U << 18U)))
                 | (1U << 16U) | (1U << 18U);

    /* AF4 = I2C1: PB8 nos bits 3:0 e PB9 nos bits 7:4 do AFR[1]. */
    GPIOB->AFR[1] = (GPIOB->AFR[1] & ~0xFFU) | 0x44U;

    /* Modo funcao alternativa (10). */
    GPIOB->MODER = (GPIOB->MODER & ~((3U << 16U) | (3U << 18U)))
                 | (2U << 16U) | (2U << 18U);

    i2c1_config();
}

/* Espera uma flag do SR1. Retorna 1 se deu timeout ou o sensor deu NACK. */
static uint8_t i2c_wait(uint32_t flag)
{
```

```
    uint32_t start = TIM2->CNT;

    while ((I2C1->SR1 & flag) == 0U)
    {
        if ((I2C1->SR1 & I2C_SR1_AF) != 0U)
            return 1U;
        if ((TIM2->CNT - start) > I2C_TIMEOUT_US)
            return 1U;
    }
    return 0U;
}

/* Em caso de erro: encerra a transferencia e limpa o flag de NACK. */
static void i2c_abort(void)
{
    I2C1->CR1 |= I2C_CR1_STOP;
    I2C1->SR1 &= ~I2C_SR1_AF;
}

/* Gera START e envia o endereco. Retorna 1 em erro. */
static uint8_t i2c_start_addr(uint8_t addr, uint8_t read)
{
    I2C1->CR1 |= I2C_CR1_START;
    if (i2c_wait(I2C_SR1_SB))
        return 1U;

    I2C1->DR = (uint8_t)((addr << 1U) | read);
    if (i2c_wait(I2C_SR1_ADDR))
        return 1U;

    return 0U;
}

/* Escreve um valor em um registrador do sensor. */
static uint8_t adxl_write_reg(uint8_t reg, uint8_t value)
{
    /* Se o barramento ficou ocupado de um erro anterior, reinicia o periferico. */
    if ((I2C1->SR2 & I2C_SR2_BUSY) != 0U)
        i2c1_config();

    if (i2c_start_addr(adxl_addr, 0U)) { i2c_abort(); return ADXL345_ERR; }
    (void)I2C1->SR1; (void)I2C1->SR2;            /* Ler SR1 e SR2 limpa o ADDR. */

    I2C1->DR = reg;
    if (i2c_wait(I2C_SR1_TXE)) { i2c_abort(); return ADXL345_ERR; }
    I2C1->DR = value;
    if (i2c_wait(I2C_SR1_BTF)) { i2c_abort(); return ADXL345_ERR; }

    I2C1->CR1 |= I2C_CR1_STOP;
    return ADXL345_OK;
}

/* Le n bytes (n = 1 ou n >= 3) a partir de um registrador, seguindo a sequencia do manual (RM0383). */
static uint8_t adxl_read_regs(uint8_t reg, uint8_t *buf, uint8_t n)
{
    uint8_t i;

    if ((I2C1->SR2 & I2C_SR2_BUSY) != 0U)
        i2c1_config();

    /* 1) Escrita: diz ao sensor qual registrador queremos ler. */
    if (i2c_start_addr(adxl_addr, 0U)) { i2c_abort(); return ADXL345_ERR; }
    (void)I2C1->SR1; (void)I2C1->SR2;
    I2C1->DR = reg;
    if (i2c_wait(I2C_SR1_BTF)) { i2c_abort(); return ADXL345_ERR; }

    /* 2) START repetido, agora em modo leitura. */
    I2C1->CR1 |= I2C_CR1_ACK;
    if (i2c_start_addr(adxl_addr, 1U)) { i2c_abort(); return ADXL345_ERR; }

    if (n == 1U)
    {
        I2C1->CR1 &= ~I2C_CR1_ACK;               /* NACK no unico byte. */
        (void)I2C1->SR1; (void)I2C1->SR2;        /* Limpa o ADDR. */
        I2C1->CR1 |= I2C_CR1_STOP;
```

```
        if (i2c_wait(I2C_SR1_RXNE)) { i2c_abort(); return ADXL345_ERR; }
        buf[0] = (uint8_t)I2C1->DR;
        return ADXL345_OK;
    }

    (void)I2C1->SR1; (void)I2C1->SR2;            /* Limpa o ADDR. */

    /* Bytes 0 .. n-4: leitura normal, com ACK. */
    for (i = 0U; i < (uint8_t)(n - 3U); i++)
    {
        if (i2c_wait(I2C_SR1_RXNE)) { i2c_abort(); return ADXL345_ERR; }
        buf[i] = (uint8_t)I2C1->DR;
    }

    /* Tres ultimos: o ultimo precisa de NACK + STOP, entao o ACK e desligado antes. */
    if (i2c_wait(I2C_SR1_BTF)) { i2c_abort(); return ADXL345_ERR; }
    I2C1->CR1 &= ~I2C_CR1_ACK;
    buf[n - 3U] = (uint8_t)I2C1->DR;
    I2C1->CR1 |= I2C_CR1_STOP;
    buf[n - 2U] = (uint8_t)I2C1->DR;
    if (i2c_wait(I2C_SR1_RXNE)) { i2c_abort(); return ADXL345_ERR; }
    buf[n - 1U] = (uint8_t)I2C1->DR;

    return ADXL345_OK;
}

uint8_t adxl345_init(void)
{
    uint8_t id = 0U;
    uint8_t found = 0U;

    i2c1_init();

    /* Procura o sensor nos dois enderecos possiveis e confere o ID (0xE5). */
    adxl_addr = ADXL_ADDR_LOW;
    if (adxl_read_regs(REG_DEVID, &id, 1U) == ADXL345_OK && id == DEVID_VALUE)
        found = 1U;
    else
    {
        adxl_addr = ADXL_ADDR_HIGH;
        if (adxl_read_regs(REG_DEVID, &id, 1U) == ADXL345_OK && id == DEVID_VALUE)
            found = 1U;
    }

    if (!found)
        return ADXL345_ERR;

    if (adxl_write_reg(REG_BW_RATE, 0x0AU)) return ADXL345_ERR;       /* 100 Hz. */
    if (adxl_write_reg(REG_DATA_FORMAT, 0x08U)) return ADXL345_ERR;   /* FULL_RES, +-2 g: 3,9 mg/LSB. */
    if (adxl_write_reg(REG_POWER_CTL, 0x08U)) return ADXL345_ERR;     /* Measure = 1: sai do standby. */

    return ADXL345_OK;
}

uint8_t adxl345_read_xyz(int16_t *x, int16_t *y, int16_t *z)
{
    uint8_t b[6];

    if (adxl_read_regs(REG_DATAX0, b, 6U) != ADXL345_OK)
        return ADXL345_ERR;

    /* Cada eixo vem em complemento de dois, byte baixo primeiro. */
    *x = (int16_t)((uint16_t)b[1] << 8U | b[0]);
    *y = (int16_t)((uint16_t)b[3] << 8U | b[2]);
    *z = (int16_t)((uint16_t)b[5] << 8U | b[4]);

    return ADXL345_OK;
}
```

A.11 Src/main.c

```
#include <stdint.h>
#include "stm32f4xx.h"
#include "lcd.h"
#include "tim.h"
```

```c
#include "keypad.h"
#include "uart.h"
#include "adxl345.h"

/* Coluna (0-15) do caractere recebido pela serial na linha 2: "NONE  X" -> 7a posicao. */
#define SERIAL_COL  6

/* Todos com 6 letras para apagar o nome anterior ao reescrever a linha 2. */
static const char *key_name[] = {"RIGHT ", "UP    ", "DOWN  ", "LEFT  ", "SELECT", "NONE  "};

static uint8_t adxl_ok = 0U;

/* Envia "X=.. (.. mg)  Y=.. (.. mg)  Z=.. (.. mg)" pela serial. Cada LSB vale 3,9 mg. */
static void write_axis(const char *name, int16_t raw)
{
    uart2_write_string(name);
    uart2_write_int(raw);
    uart2_write_string(" (");
    uart2_write_int(((int32_t)raw * 39) / 10);
    uart2_write_string(" mg)");
}

/* Comando serial 'A': le os 3 eixos do ADXL345 e responde pela serial. */
static void accel_report(void)
{
    int16_t x, y, z;

    /* Se o sensor nao respondeu no boot (ex.: fio solto), tenta de novo a cada comando. */
    if (!adxl_ok)
        adxl_ok = (adxl345_init() == ADXL345_OK);

    if (!adxl_ok || adxl345_read_xyz(&x, &y, &z) != ADXL345_OK)
    {
        adxl_ok = 0U;
        uart2_write_string("ADXL345: sem resposta (confira SCL/SDA/3V3/GND)\r\n");
        return;
    }

    write_axis("X=", x);
    write_axis("  Y=", y);
    write_axis("  Z=", z);
    uart2_write_string("\r\n");
}

int main(void)
{
    uint8_t key = KEY_NONE;
    uint8_t cand = KEY_NONE;
    uint8_t prev_cand = KEY_NONE;

    lcd_gpio_init();
    tim2_init();          /* antes de qualquer delay */
    lcd_init();
    keypad_init();
    uart2_init();
    adxl_ok = (adxl345_init() == ADXL345_OK);

    /* Tela de boas-vindas por 3 s. */
    lcd_put_cur(0, 0);
    lcd_send_string("SIST. TEMPO REAL");
    lcd_put_cur(1, 0);
    lcd_send_string("* U N I F O R *");
    delay_ms(3000U);

    /* Tela de espera de tecla. */
    lcd_clear();
    lcd_put_cur(0, 0);
    lcd_send_string("TECLA ACIONADA:");
    lcd_put_cur(1, 0);
    lcd_send_string(key_name[KEY_NONE]);

    while (1)
    {
        /* Debounce simples: so aceita a tecla se a leitura se repetir duas vezes seguidas. */
        cand = keypad_read_key();
```

```
        if (cand == prev_cand && cand != key)
        {
            key = cand;
            lcd_put_cur(1, 0);
            lcd_send_string(key_name[key]);   /* NONE ao soltar a tecla */
        }
        prev_cand = cand;

        /* Caractere recebido pela serial vai para a linha 2, depois do nome da tecla. */
        if (uart2_rx_ready())
        {
            char c = uart2_read();
            if (c >= ' ' && c <= '~')         /* ignora Enter e outros nao imprimiveis */
            {
                lcd_put_cur(1, SERIAL_COL);
                lcd_send_data((uint8_t)c);
            }

            if (c == 'A' || c == 'a')
                accel_report();
        }

        delay_ms(10U);
    }
}
```